Solution: Implement Stack using Queues

Problem Statement

Implement a stack using two queues. The stack should support standard operations like push (add an element to the top of the stack) and pop (remove an element from the top of the stack).

Examples:

1. **Input**: Push operations: [1, 2, 3], Pop operations: 2
   Output: [1]
   Explanation: After pushing 1, 2, 3 the stack looks like [1, 2, 3]. Then we perform 2 pop operations, removing 3 and 2, so the output is [1].
2. **Input**: Push operations: ["cat", "dog", "fish"], Pop operations: 1
   Output: ["cat", "dog"]
   Explanation: After pushing "cat", "dog", "fish" the stack looks like ["cat", "dog", "fish"]. Then we perform 1 pop operation, removing "fish", so the output is ["cat", "dog"].
3. **Input**: Push operations: [5, 6, 7, 8], Pop operations: 4
   Output: []
   Explanation: After pushing 5, 6, 7, 8 the stack looks like [5, 6, 7, 8]. After performing 4 pop operations we are left with zero elements in the stack [].

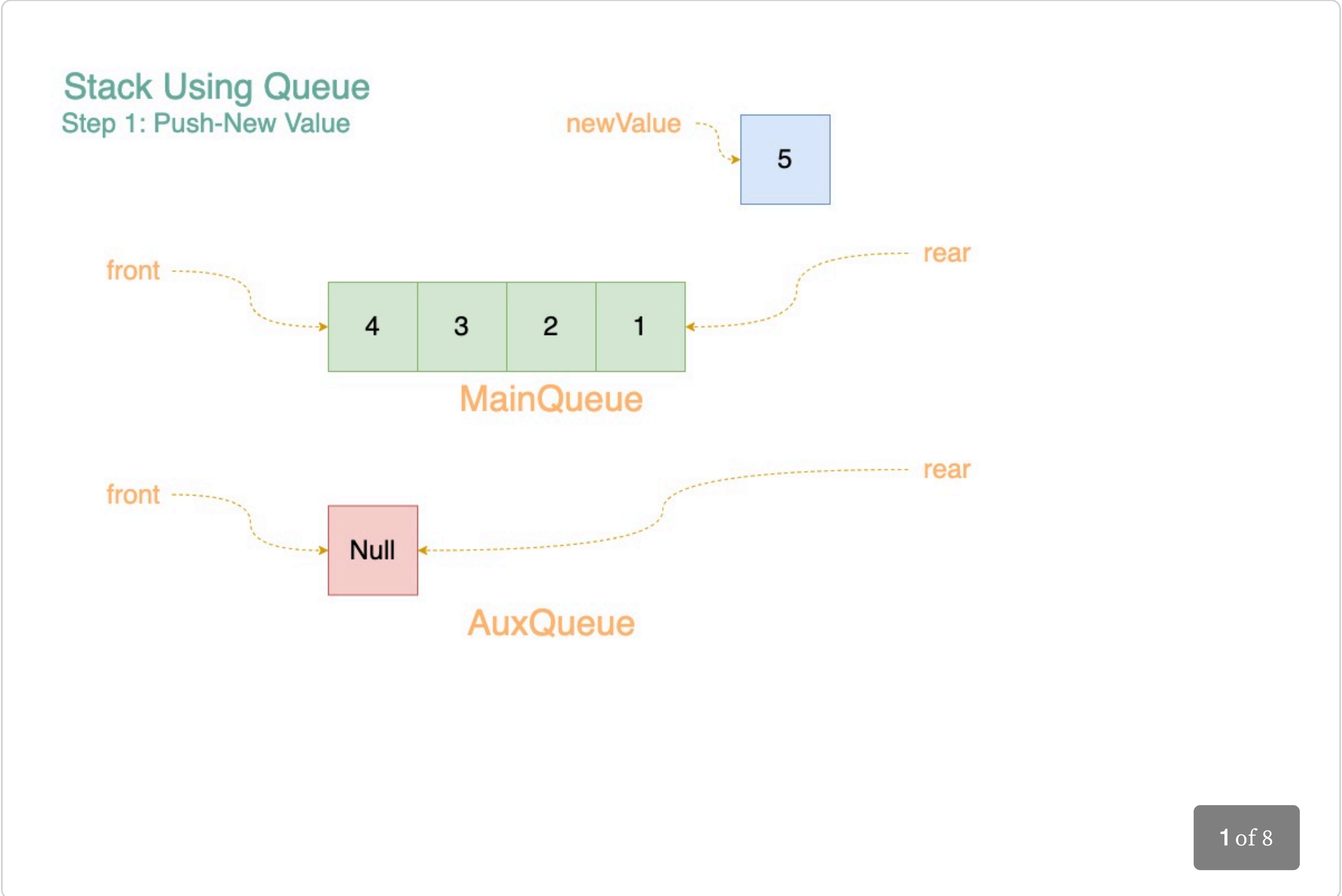
Solution

Solution Explanation in Paragraph Format:

To implement a stack using two queues, we use the principle of reversing the order of elements. When a new element is pushed onto the stack, it needs to be positioned such that it can be accessed first, adhering to the stack's Last-In-First-Out (LIFO) nature. This is achieved by using two queues, named main and auxiliary queues. Each new element is first added to the auxiliary queue. Then, all elements from the main queue (which holds the stack elements) are dequeued and enqueued into the auxiliary queue. This process effectively places the newest element at the front of the queue. Finally, the two queues are swapped, making the auxiliary queue the new main queue and vice versa. The pop operation is straightforward, involving simply dequeuing an element from the main queue, which is now at the front due to the previous rearrangements.

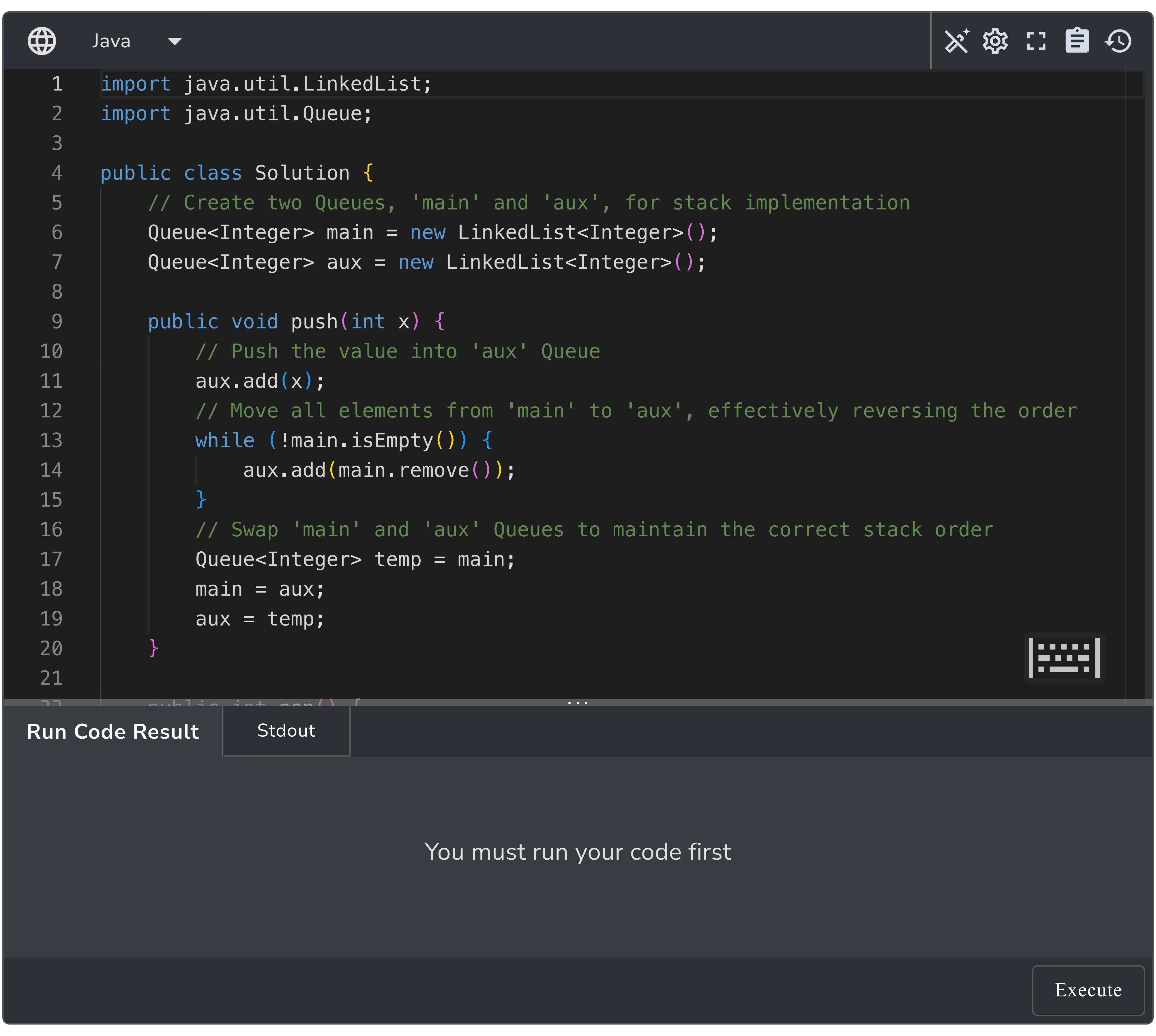
Step-by-Step Solution:

1. **Initialize Two Queues**: Create two queues (`main` and `aux`) to manage the stack elements.
2. **Push Operation**:
   - Add the new element to the `aux` queue.
   - Move all elements from the `main` queue to the `aux` queue, thereby placing the new element at the front of the combined queue.
   - Swap the roles of `main` and `aux` queues, making the auxiliary queue the new main queue.
3. **Pop Operation**:
   - Remove and return the front element from the `main` queue, which is effectively the top of the stack.



Code

Here is how we can implement this algorithm:



```java
import java.util.LinkedList;
import java.util.Queue;

public class Solution {
    // Create two Queues, 'main' and 'aux', for stack implementation
    Queue<Integer> main = new LinkedList<Integer>();
    Queue<Integer> aux = new LinkedList<Integer>();

    public void push(int x) {
        // Push the value into 'aux' Queue
        aux.add(x);
        // Move all elements from 'main' to 'aux', effectively reversing the order
        while (!main.isEmpty()) {
            aux.add(main.remove());
        }
        // Swap 'main' and 'aux' Queues to maintain the correct stack order
        Queue<Integer> temp = main;
        main = aux;
        aux = temp;
    }

```

You must run your code first

Time and Space Complexity

The time complexity for the push operation is O(n) because we're transferring all elements from the main queue to the auxiliary queue. The pop operation is O(1) because we're just dequeuing from the main queue.

The space complexity is O(n), where n is the number of elements in the stack. This is because we store all elements in the main and/or auxiliary queues.

← Previous
Problem 2: Implement Stack Using Queues

Next →
Problem 3: Generate Binary Numbers From 1 To N

☐ Mark as Completed